Memo: Tenacious Sales Agent — Fine-Tuning Results and Production Recommendation

To: CEO, CFO — Tenacious
From: AI/ML Engineering
Date: 2026-05-02
Re: Tenacious-Bench v0.1 evaluation results and deployment decision

Page 1 — The Decision

What Was Built

We built two artifacts:

1. **Tenacious-Bench v0.1** — a 230-task evaluation benchmark that measures the five failure modes identified in the Week 10 audit (tone drift, missing signal grounding, formulaic openers, trajectory inconsistency, constraint violations). The benchmark uses seven machine-verifiable rubric dimensions with no human judgment required per scoring run. Cost to build: **$0.06**.
2. **A LoRA fine-tuned adapter** — Qwen3-4B trained via ORPO (Odds Ratio Preference Optimization) on 381 preference pairs derived from the training partition. The adapter is 66M parameters (1.6% of the 4B base model), loads in under 60 seconds on a T4 GPU, and can be swapped into the existing inference stack without retraining the base model.

Headline Result

Arm	Score / 5.0	95% CI
Week 10 baseline	4.008	[3.668, 4.301]
Prompt-engineered (no training)	4.172	[3.887, 4.449]
Trained adapter (ORPO)	**4.462**	**[4.193, 4.704]**

- **Delta A: +0.454 improvement over Week 10 baseline, p=0.001** (primary metric, required p<0.05)
- **Delta B: +0.290 improvement over prompt engineering alone, p=0.021** — training adds value beyond what a careful system prompt can achieve
- Largest gain on `tone_drift` (+0.679) — the failure mode with the highest Week 10 frequency (38%)

Cost Per Task Delta

	Cost/task	Avg latency
Week 10 baseline (base model API)	$0.0004	4.6s
Trained adapter (LoRA on-device)	$0.00024	9.1s

LoRA inference costs **40% less per task** due to local generation vs API routing. Latency approximately doubles (4.6s → 9.1s per message) because generation moves from a high-throughput API to a T4 GPU. At Tenacious's current volume this is acceptable; at 10× scale a dedicated inference endpoint is warranted.

Production Recommendation

Deploy the LoRA adapter to the production Tenacious sales agent. Specific actions:

1. Push the adapter to HuggingFace and integrate into the inference pipeline with `FastLanguageModel.for_inference()` (one-line swap).
2. Add a weekly Tenacious-Bench dev-split evaluation to CI — score 10 sampled outputs per run; alert if mean drops below 4.0/5.0.
3. Re-evaluate in 90 days with fresh held-out tasks drawn from new probes.

Page 2 — Skeptic's Appendix

Failure Modes the New Benchmark Still Does Not Capture

1. Multi-turn conversation coherence beyond single-exchange. The current `trajectory` checker uses a single-turn proxy (does the response acknowledge an objection?). It does not test whether the agent maintains consistent tone and factual claims across a 5-turn negotiation. We know from trace_018 that the agent can contradict itself across turns — this is not yet measured.

2. Prospect reply tone analysis. Tenacious-Bench measures what the agent sends, not how prospects respond. High rubric scores do not guarantee replies or booked meetings. The benchmark is a leading indicator, not a conversion proxy.

3. Segment-specific signal weighting. The rubric applies the same dimension weights across SMB, Series B, and Enterprise tasks. In practice, an Enterprise AE weights trajectory and constraint compliance differently than an SMB rep does. Segment-stratified weights are not yet implemented.

Public-Signal Lossiness in the Ground Truth

All 108 public-signal tasks in the dataset use signals sourced from public data (LinkedIn, Crunchbase, job boards) with a 30–90 day publication lag. A signal that was accurate at task-authoring time (e.g., "Series C $45M in Q2 2023") may be stale by the time the agent uses it. The benchmark checks that the agent *references* a signal; it does not check whether the signal is still actionable.

One Honest Unresolved Failure

The `trajectory` failure mode showed the smallest improvement across all four categories: **Δ=+0.154** (trained 3.554 vs baseline 3.400). The preference pairs targeting this failure mode used a single-exchange proxy, not real multi-turn conversation data. The model learned to add acknowledgment phrases ("that makes sense") but did not demonstrably improve on maintaining factual consistency across turns. This is the one area where we cannot claim the fine-tuning solved the underlying problem.

Kill-Switch Trigger

Revert to the Week 10 base model if either condition is met on a 100-task rolling production window:

- **Banned phrase rate > 5%** — any resurgence of prohibited language in live outputs
- **Tenacious-Bench dev mean < 3.8/5.0** — a regression below the pre-training baseline (4.008 – 0.2 buffer)

Both checks are automatable using `evaluation/scoring_evaluator.py --split dev --mock-llm` with no API cost.